tp

User Guide - Ledger 67

Table of Contents

- User Guide - Ledger 67
  - Table of Contents
  - 1. Introduction
  - 2. Quick Start
    - 2.1 Prerequisites
    - 2.2 Getting Started
  - 3. Understanding Double-Entry Accounting
    - 3.1 What is Double-Entry Bookkeeping?
    - 3.2 Hierarchical Accounts
    - 3.3 Sign Convention
    - 3.4 Monetary Precision
    - 3.5 Key Concepts for Beginners
  - 4. Features
    - 4.1 UI Assist: `uiassist`
    - 4.2 Adding a Transaction: `add`
    - 4.3 Listing and Filtering Transactions: `list`
    - 4.4 Editing a Transaction: `edit`
    - 4.5 Deleting Transactions: `delete`
    - 4.6 Clearing All Transactions: `clear`
    - 4.7 Currency Conversion: `convert`
    - 4.8 Refreshing Exchange Rates: `rates`
    - 4.9 Confirming Converted Transactions: `confirm`
    - 4.10 Viewing the Balance Sheet: `balance`
    - 4.11 Getting Help: `help`
  - 5. FAQ
  - 6. Command Summary
  - 7. Tips for Effective Use

1. Introduction

Ledger67 is a command-line double-entry accounting system designed to help individuals and small businesses manage their financial transactions efficiently. Built on the principles of double-entry bookkeeping, Ledger67 ensures that every financial transaction is recorded with both debit and credit entries, maintaining the fundamental accounting equation: **Assets = Liabilities + Equity**.

Whether you're a student learning accounting, a small business owner, or someone who wants to better manage personal finances, Ledger67 provides a simple yet powerful way to track your financial activities with proper accounting rigor.

Ledger67 also provides balance sheet generation and export features to help users analyse their financial position in real time.

2. Quick Start

2.1 Prerequisites

1. Ensure that you have **Java 17 or above** installed on your computer.
2. Download the latest version of `Ledger67` from the project repository.

2.2 Getting Started

Option 1: Using the JAR file (Simplest)

1. **Download the application**: Obtain the `tp.jar` file from the latest release.
2. **Run the application**: Open a terminal and navigate to the directory containing `tp.jar`, then run:

   ```
   java -jar ledger67.jar
   ```

Option 2: Using Gradle (For Developers)

If you have the source code and want to build from source:

1. **Navigate to project directory**:

   ```
   cd /path/to/ledger67
   ```

2. **Build the project using Gradle**:

   ```
   ./gradlew build
   ```

3. **Run the application**:

   ```
   ./gradlew run
   ```

Or to create a runnable JAR:

```
./gradlew shadowJar
java -jar build/libs/ledger67.jar
```

4. **Start recording transactions**: Use the commands below to add, view, edit, and manage your financial transactions.

3. Understanding Double-Entry Accounting

3.1 What is Double-Entry Bookkeeping?

Double-entry bookkeeping is an accounting method where every financial transaction affects at least two accounts. For every debit entry, there must be a corresponding credit entry of equal value. This system ensures that the accounting equation always remains balanced.

3.2 Hierarchical Accounts

Ledger67 supports hierarchical account structures using `:`.

This allows accounts to be organised into categories and subcategories:

- `Assets:Cash`
- `Assets:Bank:DBS`
- `Expenses:Food`

This structure enables more powerful filtering and organisation of financial data.

3.3 Sign Convention

Ledger67 uses a **signed amount model** instead of debit/credit.

- `+` indicates an **increase**
- `-` indicates a **decrease**

Recommended Usage

Account Type	Increase	Decrease
Assets, Expenses	+	-
Liabilities, Equity, Income	-	+

Note

Ledger67 only checks that transactions are **balanced**, and does not enforce sign conventions. Users are encouraged to follow the recommended usage for consistency.

3.4 Monetary Precision

Ledger67 stores monetary values using **2 decimal places**, which is standard for currency amounts.

If users enter values with more than 2 decimal places, Ledger67 will round them before storing them.

Examples:

- `100.999999` is stored as `101.00`
- `50.1` is stored as `50.10`
- `75.555` is stored as `75.56`

Users should therefore avoid entering excessive decimal precision if exact sub-cent values are important.

3.5 Key Concepts for Beginners

1. The Accounting Equation

```
Assets = Liabilities + Equity

Where:
Equity is affected by Income and Expenses:
Equity = Initial Equity + (Income - Expenses)
```

- **Assets**: What you own (cash, inventory, equipment)
- **Liabilities**: What you owe (loans, accounts payable)
- **Equity**: Your ownership interest in the business
- **Income**: Money received in exchange for labor or the sale of products
- **Expenses**: Cost of Operation

2. How Transactions Work in Ledger67

In Ledger67, each transaction in Ledger67 consists of multiple postings that together form a complete double-entry record.

The system ensures that all postings within a transaction are balanced, meaning the total debits equal total credits.

Each transaction is a collection of `Postings` Postings are records of money going into each category (Assets, Liabilities, Equity, etc) These categories can have subcategories such as Assets:Cash or Liabilities:Loans These `Postings` must balance each other out.

3. Example: Recording a Sale

When you make a sale for $100 cash:

- `Assets:Cash` increases by $100
- `Income` increases by $100

The equation is balanced and the transaction is valid. In Ledger67, you would record this as a single transaction, and the system understands the double-entry implications.

4. Features

4.1 UI Assist: `uiassist`

Toggles whether the user is using UI Assist in which the program helps the user guide through the input or the user is using the standard tagging system

Format: `uiassist -on/-off` `uiassist -on` toggles the UI Assistance on. The user is now using prompted inputs to fill in the details `uiassist -off` toggles the UI Assistance off. The user is now using the standard tagging system

4.2 Adding a Transaction: `add`

Adds a new financial transaction to your ledger.

Format (Manual): `add -date DATE -desc DESCRIPTION -p POSTING1 -p POSTING2 -c CURRENCY`

Parameters

- `-date` : The date of the transaction (e.g., `DD/MM/YYYY`).
- `-desc` : A brief description of the transaction.
- `-p` : A posting containing an account name and an amount (enclosed in quotes). You must have at least two postings.
- `-c` : The currency code (e.g., SGD, USD, EUR).

Example

```
add -date 18/03/2026 -desc "Office supplies"
-p "Assets:Cash -45.50"
-p "Expenses:OfficeSupplies 45.50"
-c SGD
```

Format (Presets): `add -date DATE -preset TYPE AMOUNT -c CURRENCY`

Parameters

- `-date` : The date of the transaction (e.g., `DD/MM/YYYY`).
- `-desc` : A brief description of the transaction.
- `-presets` : Presets: DAILYEXPENSE, INCOME, BUYINGSTOCKS
- `-c` : The currency code (e.g., SGD, USD, EUR).

Notes

- Account roots are not case-sensitive
- Sub-account names are case-sensitive, this is to preserve acronyms and other valid mixed-cased names.
- Amounts are stored using standard currency precision of **2 decimal places**.
- If an amount is entered with more than 2 decimal places, Ledger67 will round it to 2 decimal places before storing it.

Example

```
add -date 18/03/2026 -preset DAILYEXPENSE 50.00 -c SGD
```

Example Output

```
Enter Command: add -date 18/03/2026 -preset DAILYEXPENSE 50.00 -c SGD
Transaction added successfully via preset.
======================================================================
Enter Command: list
ID: 4 | Date: 18/03/2026 | Desc: DAILYEXPENSE | [SGD]
    Expenses                        :      50.00
    Assets:Cash                     :     -50.00
======================================================================
```

4.3 Listing and Filtering Transactions: `list`

Displays recorded transactions. You can view all transactions or use filters to narrow down the results by date, account, keyword, or currency.

Format: `list [-acc ACCOUNT] [-begin DATE] [-end DATE] [-match REGEX] [-to CURRENCY]`

Parameters:

- `-acc ACCOUNT` : Only show transactions containing this account (and only show that specific account's line).
- `-begin DATE` : Show transactions on or after this date (DD/MM/YYYY).

- `-end DATE` : Show transactions on or before this date (DD/MM/YYYY).
- `-match REGEX` : Show transactions where the description matches the given keyword or regular expression.
- `-to CURRENCY` : Display all values converted to a specific currency for viewing purposes.

> **Note on Currency**: When using `-to` , the displayed values are **view-only**. To permanently save these converted values to your ledger, follow up with the `confirm` command.

Examples:

- **View everything**: `list`
- **View January food expenses**: `list -acc Expenses:Food -begin 01/01/2026 -end 31/01/2026`
- **Search for any "Lunch" or "Dinner"**: `list -match "(Lunch|Dinner)"`
- **View all expenses in USD**: `list -acc Expenses -to USD`

Example Output

```
ID: 2 | Date: 2026-03-19 | Desc: Salary | [USD]
Assets:Bank:DBS              :    3000.00

ID: 3 | Date: 2026-03-20 | Desc: Transfer | [SGD]
Assets:Cash                  :    -500.00
```

Combining with Currency View

You can combine filtering with currency display:

```
list -acc Assets -to USD
```

This will:

- Filter transactions under `Assets`
- Display converted values in USD

Notes

- This is a **view feature** — it does not modify stored data.
- Filtering works based on account hierarchy (not simple text matching).

View Converted Values of All Listed Transactions: `list -to TARGET_CURRENCY`

Views all existing transactions to a different currency.

Format: `list -to TARGET_CURRENCY`

Parameters:

- `-to TARGET_CURRENCY` : Target currency code - `SGD` , `USD` , or `EUR`

Example:

```
list -to SGD
```

Output:

```
ID: 1 | Date: 18/03/2026 | Desc: Office supplies | [SGD -> USD]
    Assets:Cash                    :     -45.50 | Display: -33.97 USD
    Expenses:OfficeSupplies        :      45.50 | Display: 33.97 USD
```

Note

- This is a view-mode feature only. It does NOT overwrite the stored transaction currency or amount.
- To store conversions:
  - Use `confirm all` to store all transactions
  - Use `confirm ID` to store a specific transaction

4.4 Editing a Transaction: `edit`

Modifies an existing transaction.

Format: `edit ID [-date DATE] [-desc DESC] [-p POSTING] [-c CURRENCY]`

Parameters:

- `ID` : The transaction ID to edit (shown in `list` command)
- Field updates (any combination):
  - `-date NEW_DATE` : Update transaction date
  - `-desc NEW_DESCRIPTION` : Update description
  - `-p POSTING` : New `Postings` that replaces old ones. At least 2. Transaction balance is checked.
  - `-c NEW_CURRENCY` : Update currency

Examples:

```
edit 1 -desc "Office stationery purchase"
edit 2 -c EUR
```

Example Output

```
Enter Command: list
ID: 4 | Date: 18/03/2026 | Desc: DAILYEXPENSE | [SGD]
    Expenses                        :      50.00
    Assets:Cash                     :     -50.00
======================================================================
Enter Command: edit 4 -c EUR
Transaction edited successfully.
======================================================================
Enter Command: list
ID: 4 | Date: 18/03/2026 | Desc: DAILYEXPENSE | [EUR]
    Expenses                        :      50.00
    Assets:Cash                     :     -50.00
======================================================================
```

Note

- Edited posting amounts are also stored to **2 decimal places**.
- If more than 2 decimal places are entered, Ledger67 will round them before saving.

4.5 Deleting Transactions: `delete`

Removes transactions from the ledger. You can delete a single transaction by its ID or remove multiple transactions at once using filters.

Single Deletion

Format: `delete ID`

- **Example**: `delete 3`

Bulk Deletion

Format: `delete [-begin DATE] [-end DATE] [-match REGEX] [-acc ACCOUNT]`

Parameters:

- `-begin DATE` : Delete transactions starting from this date (DD/MM/YYYY).
- `-end DATE` : Delete transactions up to this date (DD/MM/YYYY).
- `-match REGEX` : Delete transactions matching a specific keyword/description.

- `-acc ACCOUNT` : Delete transactions that involve a specific account.

Examples:

- **Delete by Keyword**: `delete -match Steak` (Removes all transactions with "Steak" in the description).
- **Delete by Date Range**: `delete -begin 01/01/2026 -end 15/01/2026` (Removes everything in the first half of January).
- **Layered Deletion**: `delete -acc Expenses:Entertainment -match Netflix` (Removes Netflix transactions specifically from that account).

Example Output

```
Enter Command: list
ID: 1 | Date: 18/03/2026 | Desc: Office supplies | [SGD]
    Assets:Cash                    :     -45.50
    Expenses:OfficeSupplies        :      45.50
ID: 2 | Date: 18/03/2026 | Desc: Office supplies | [SGD]
    Assets:Cash                    :     -45.50
    Expenses:OfficeSupplies        :      45.50
ID: 3 | Date: 18/03/2026 | Desc: Office supplies | [SGD]
    Assets:Cash                    :     -45.50
    Expenses:OfficeSupplies        :      45.50
======================================================================
Enter Command: delete -match Office
Successfully deleted 3 transaction(s) matching criteria.
======================================================================
Enter Command: list
No matching transactions found.
```

4.6 Clearing All Transactions: `clear`

Removes all transactions from the ledger (use with caution!).

Format: `clear`

Example:

```
clear
```

Note

- IDs will increase even after clear to keep to ledger semantics.

4.7 Currency Conversion: `convert`

Converts amounts between different currencies using current exchange rates.

Format: `convert -a AMOUNT -from SOURCE_CURRENCY -to TARGET_CURRENCY`

Parameters:

- `-a AMOUNT` : Amount to convert (positive number)
- `-from SOURCE_CURRENCY` : Source currency code - `SGD` , `USD` , or `EUR`
- `-to TARGET_CURRENCY` : Target currency code - `SGD` , `USD` , or `EUR`

Example:

```
convert -a 100 -from USD -to SGD
```

Output:

```
100.00 USD = 135.50 SGD
```

Note

- This is a view-mode feature only. It does NOT overwrite the stored transaction currency or amount.

Converting Existing Transactions: `convert transaction`

Converts an existing transaction to a different currency.

Format: `convert transaction ID -to TARGET_CURRENCY`

Parameters:

- `ID` : The transaction ID to convert (shown in `list` command)
- `-to TARGET_CURRENCY` : Target currency code - `SGD` , `USD` , or `EUR`

Example:

```
convert transaction 3 -to SGD
```

Output:

```
Transaction 3: 50.00 USD = 67.75 SGD
```

Note

- This is a view-mode feature only. It does NOT overwrite the stored transaction currency or amount.
- To store the converted value, use the `confirm` command.

4.8 Refreshing Exchange Rates: `rates`

Refreshes live exchange rates from external sources.

Format: `rates refresh`

Example:

```
rates refresh
```

Output:

```
Exchange rates refreshed successfully for 2026-03-19.
```

4.9 Confirming Converted Transactions: `confirm`

After using conversion commands, Ledger67 allows you to **store the converted values permanently**.

By default, all conversions are **view-only**. You must explicitly confirm to update the stored transaction(s).

Confirming a Single Converted Transaction

After using:

```
convert transaction ID -to TARGET_CURRENCY
```

You can store the converted result using:

```
confirm
```

Example:

```
convert transaction 3 -to SGD
confirm
```

Result:

- The transaction's **amount** is updated to the converted value
- The transaction's **currency** is updated to the target currency
- Changes are saved permanently

Confirming Transactions from List View

After using:

```
list transaction -to TARGET_CURRENCY
```

You can choose to store:

a) All transactions

```
confirm all
```

b) A specific transaction

```
confirm ID
```

Example:

```
list transaction -to SGD
confirm 2
```

Important Notes

- If you enter any command **other than** `confirm` , the conversion will be **ignored**
- Confirmation updates:
  - transaction **amount**
  - transaction **currency**
- Changes are **saved to storage immediately**

4.10 Viewing the Balance Sheet: `balance`

Displays a summary of your financial position based on the accounting equation:

```
Assets = Liabilities + Equity
```

This command aggregates all transactions and shows totals by account category.

Basic Format

```
balance
```

Example Output

```
===== BALANCE SHEET =====
Scope: ALL ACCOUNTS
Report Currency: ORIGINAL TRANSACTION CURRENCIES

ASSETS
Assets:Cash                          350.00
Assets:Bank:Main                    200.00
Total Assets                        550.00

LIABILITIES
Total Liabilities                     0.00

EQUITY
Equity:Capital                      400.00
Current Period Net Income           150.00
Total Equity                        550.00

CHECK
Total Assets                        550.00
Liabilities + Total Equity          550.00
Equation Status                     PASS
========================================
```

Including Income and Expenses

Ledger67 automatically incorporates:

- **Income** → increases Equity
- **Expenses** → decreases Equity

These are reflected as:

```
Net Income = Income - Expenses
```

This ensures the balance sheet satisfies:

```
Assets = Liabilities + Equity
```

Filtering Balance Sheet by Account: `balance -acc`

Allows you to generate a balance sheet for a specific account hierarchy.

Format

```
balance -acc ACCOUNT
```

Examples

```
balance -acc Assets
balance -acc Assets:Bank
```

Behaviour

- Includes only accounts under the specified hierarchy
- Uses the same hierarchical logic as `list -acc`
- May not balance fully (partial view)

Example Output

```
Scope: Assets:Bank
Equation Status: FILTERED VIEW (may not balance)
```

Viewing Balance Sheet in Another Currency: `balance -to`

Displays all values converted into a target currency.

Format

```
balance -to TARGET_CURRENCY
```

Example

```
balance -to USD
```

Behaviour

- Converts all postings into the selected currency
- Uses latest exchange rates
- Does NOT modify stored data

Combining Filters and Conversion

You can combine both options:

```
balance -acc Assets:Bank -to USD
```

Exporting Balance Sheet

Each time the `balance` command is run:

- A CSV file is generated at:

  `data/balance-sheet.csv`

File Contents

The CSV includes:

- Account names
- Aggregated balances

- Report currency

Notes

- The file is overwritten each time `balance` is executed
- Useful for Excel analysis or reporting
- Export does NOT affect ledger data

Important Notes

- `balance` is a **read-only feature**
- It does NOT modify any transactions
- Currency conversion is **view-only**
- Filtering may result in non-balanced views

4.11 Getting Help: `help`

Displays available commands and usage instructions.

Format: `help`

Example:

```
help
```

5. FAQ

Q: How do I transfer my data to another computer?

A: Ledger67 stores transactions locally in a file. You can transfer your data by copying the data file to another computer.

Q: What's the difference between debit and credit in simple terms?

A: Think of debit as "what comes in" or "expenses/increases in assets," and credit as "what goes out" or "income/increases in liabilities or equity." In Ledger67, you record transactions based on how they affect your accounts.

Q: Can I use Ledger67 for personal finance tracking?

A: Absolutely! While based on double-entry principles used in business accounting, Ledger67 works perfectly for personal finance. Record expenses as debits and income as credits to maintain

a clear picture of your financial health.

Q: What currencies does Ledger67 support?

A: Currently, Ledger67 supports SGD (Singapore Dollar), USD (US Dollar), and EUR (Euro). More currencies can be added in future updates.

Q: How does Ledger67 handle the double-entry aspect if I only enter one side?

A: Ledger67 follows a strict double-entry system.

Every transaction must include at least two postings, and the system ensures that:

- Total debits = total credits

Transactions that do not balance will be rejected.

Q: How does Ledger67 handle duplicate single value flags?

A: If a duplicate single-value flag is provided, the first input will be chosen.

6. Command Summary

Command	Format	Description
Add	`add -date DATE -desc DESC -p POSTING -p POSTING -c CURRENCY`	Add a new transaction
List	`list`	Display all transactions
Filter by Account	`list -acc ACCOUNT`	Filter transactions by account
List with Conversion	`list -to TARGET_CURRENCY`	View transactions in another currency
Filter + Convert	`list -acc ACCOUNT -to TARGET_CURRENCY`	Filter and convert
Edit	`edit ID [-date DATE] [-desc DESC] [-p POSTING] [-c CURRENCY]`	Edit an existing transaction
Delete	`delete ID`	Remove a transaction
Clear	`clear`	Remove all transactions
Convert	`convert -a AMOUNT -from SOURCE -to TARGET`	Convert currencies
Convert Transaction	`convert transaction ID -to TARGET`	Convert stored transaction

Command	Format	Description
Rates	`rates refresh`	Refresh exchange rates
Confirm	`confirm` , `confirm all` , `confirm ID`	Store converted transactions
Balance Sheet	`balance`	View full balance sheet
Balance (Filter)	`balance -acc ACCOUNT`	Filter balance sheet by account
Balance (Convert)	`balance -to TARGET_CURRENCY`	View balance in another currency
Balance (Combined)	`balance -acc ACCOUNT -to TARGET_CURRENCY`	Filter + convert balance
Help	`help`	Show help
Exit	`exit`	Exit program

7. Tips for Effective Use

1. **Be Consistent**: Use clear, descriptive transaction descriptions that you'll understand later.
2. **Regular Updates**: Record transactions regularly to maintain accurate financial records.
3. **Review Periodically**: Use the `list` command weekly or monthly to review your financial position.
4. **Backup Important Data**: Since transactions are stored in memory, consider exporting or noting down important transactions.
5. **Learn the Basics**: Understanding basic accounting principles will help you get the most out of Ledger67.
6. **Currency Conversion**: Use the view and confirm feature efficiently to better manage multiple currencies,

Ledger67 - Simplifying double-entry accounting for everyone